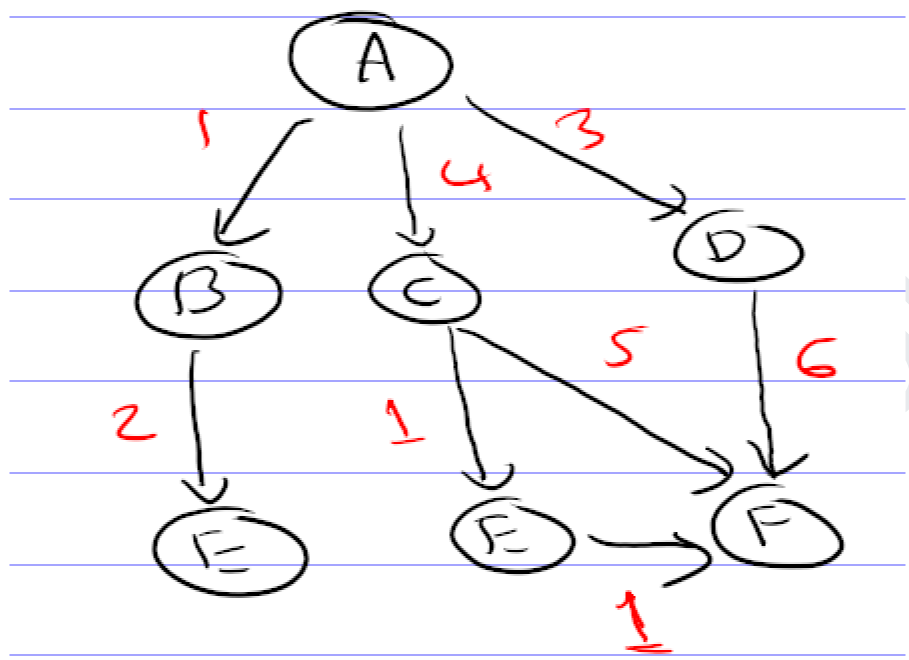
Lab 5

Write a basic program to implement A* Search and find the path for the given problem

Example problem



Nodes	Heuristic
A (initial state)	7
B	6
C	2
D	4
E	1
F (goal state)	0

Here, A is the initial state and F is the goal state

<u>Pseudocode</u>

BEGIN

```
// Define graph as an adjacency list with cost to neighbors
DEFINE Graph as:
  A → [(B, 1), (C, 4), (D, 3)]
  B → [(E, 2)]
  C → [(E, 1), (F, 5)]
  D → [(F, 6)]
  E → [(F, 1)]
  F → []

// Define heuristic estimates from each node to goal 'F'
DEFINE Heuristic as:
  h(A) = 7
  h(B) = 6
  h(C) = 2
  h(D) = 4
  h(E) = 1
  h(F) = 0

FUNCTION A_Star_Search(start, goal):
  INITIALIZE priority queue open_set
  CALCULATE f(start) = 0 + h(start)
  INSERT (f, g=0, node=start, path=[start]) into open_set

  INITIALIZE visited as empty set

  WHILE open_set is not empty:
    REMOVE node with smallest f from open_set → (f, g, current_node, path)

    IF current_node = goal THEN
      RETURN path  // Goal reached

    IF current_node in visited THEN
      CONTINUE to next iteration

    ADD current_node to visited

    FOR each neighbor and cost in Graph[current_node]:
      IF neighbor not in visited THEN
        COMPUTE new_g = g + cost
        LOOKUP h(neighbor) from Heuristic
        COMPUTE new_f = new_g + h(neighbor)
        INSERT (new_f, new_g, neighbor, path + [neighbor]) into open_set
```

```
  RETURN None  // No path found

 FUNCTION Main:
  DISPLAY "Graph adjacency list with costs:"
  FOR each node in Graph:
   DISPLAY node → neighbors

  SET start = 'A'
  SET goal = 'F'

  CALL A_Star_Search(start, goal) → path

  IF path exists THEN
   DISPLAY "Shortest path from start to goal using A* Search:"
   DISPLAY path
  ELSE
   DISPLAY "No path found from start to goal."

 CALL Main

END
```

Report format

Title: Write a program ……..
Date: Lab assigned date

Objective

- To implement the A* Search algorithm using an adjacency list to explore nodes in a graph or tree structure.
- To find and display the path from a given start node to a destination node using Greedy A* Search traversal.

Theory

- A* Search Algorithm

Implementation

- Write down the problem statement
- Python Code

- Output (Screenshots of your output)

Conclusion